

Fakultät für Informatik
Universität Duisburg-Essen

Bachelorarbeit

How to Write and Structure your Thesis that has an Experiment^{v 20201411}

Your Name

Matrikelnummer: 123456
Studiengang: Angewandte Informatik

Essen, 10.10.2020

Betreuer: Stefan Udo Hanenberg

1. Gutachter: Blablabla

2. Gutachter: <Name des 2. Gutachters>

Abstract

Write here your English summary.

Zusammenfassung

Your abstract explains in at most half a page what the goal of your work is

Contents

List of Figures	iv
List of Tables	v
List of Abbreviations	vi
1 Introduction	1
1.1 Research question	2
1.2 Structure of this work	2
2 Background	4
2.1 Your studied technique	4
2.2 Randomized control trials	4
3 Related work	6
3.1 Studies on the language construct or similar constructs	6
3.2 Studies on stuff that influenced your experiment	6
3.3 Studies, Metastudies, etc. that share similarities to yours	7
4 Baseline Indentation Experiment	8
4.1 Initial Considerations	8
4.2 Experiment Layout	9
4.3 Experiment Design	9
4.4 Analysis	10
4.5 Discussion	12
5 Commonalities between Preliminary Experiments	13
5.1 Initial Considerations	13
5.2 Measurement and Conduction	14
6 Chained Conditions Preliminary Experiment	15
6.1 Initial Considerations	15
6.2 Experiment Layout	15
6.3 Experiment Design	16
6.4 Analysis	16
6.5 Discussion	17
7 Dynamic Indentation Preliminary Experiment	19
7.1 Initial Considerations	19
7.2 Experiment Layout	19
7.3 Experiment Design	20
7.4 Analysis	21

7.5	Discussion	22
8	Overlap Preliminary Experiment	23
8.1	Initial Considerations	23
8.2	Experiment Layout	24
8.3	Experiment Design	24
8.4	Analysis	26
8.5	Discussion	27
9	Main Experiment	29
9.1	Initial considerations	29
9.2	Hypotheses	30
9.3	Measurement technique	30
9.4	Experimental design	31
9.4.1	Teaching and Teaching Material	31
9.4.2	Tasks	31
9.5	Experimental layout	31
9.6	Experimental protocol	32
9.7	Experiment execution	32
9.8	Results	33
9.8.1	Measurements and Data Description	33
9.8.2	Statistical Analysis	35
9.9	Threats to Validity	36
9.10	Summary and discussion	36
10	Summary and Discussion	37
11	Conclusion	38
12	How References Work	39
	References	39
A	Appendix	42
A.1	Experiment Summary	42
A.2	Handling Tables and Table Formats in Lyx	43
A.2.1	Standard Table	43
A.2.2	Long Table or Figure Descriptions	43
A.2.3	Highlighting Stuff	45

List of Figures

1	Example for How to write Hello World in Java (only this appears in the index!)	4
2	Example code snippet with nesting depth 9, indentation length 2 and the highlighted condition marks the target if-statement on nesting level 5.	10
3	Interaction diagram Indentation_Length $\times$ Nesting_Level	11
4	Example code snippet with nesting depth 9, indentation length 2 and the highlighted condition marks the target if-statement on nesting level 7.	17
5	Shortened example code snippet with dynamic indentation. The highlighted condition is on nesting level 1 with a condition length of 2.	20
6	Example code snippet in the Noise condition with a indentation length of 6. The highlighted condition is located at nesting level 3 and visually overlaps with the longer words at the neighboring nesting levels above and below.	25
7	Interaction diagram Indentation_Length $\times$ Overlap	27
8	Boxplot for measured data	34

List of Tables

1	Experiment results for the Baseline Indentation Experiment . . .	11
2	Experiment results for the Indentation Length by Nesting Level Experiment	18
3	Experiment results for the Nesting Level by Condition Word Length Experiment	21
4	Experiment results for the Indentation Length by Overlap by Nesting Level Experiment	26
5	Experiment layout – main experiment	32
6	Experiment layout – main experiment	33
7	Descriptive statistics of measured data	34
8	Results – ANOVA	35
9	Main Experiment – means and confidence intervals	36
10	Experiment design summary - main experiment	42
11	Experiment execution summary - main experiment	43
12	Centralized table with dynamic column size	44
13	Centralized table with row breaks (requires fixed sizes column size)	44
14	Sometimes it is nicer to resize the table so that it matches on a page	44

List of Abbreviations

The next list describes several abbreviations that will be later used within the body of the document

ANOVA	analysis of variance
CI	confidence interval
M	mean (arithmetic mean)
N	Number of data sets (subjects)

1 Introduction

The goal of this template is to guide you through the creation of your thesis. A bunch of people did already their theses with me and it turned out that they often have comparable questions (and my answers are comparable, too). Hence, this document should help you understanding how the structure of your thesis should look like and what parts of your thesis belong to what parts in the document. And in case you consider this document as not sufficient: contact your supervisor (who is probably me: <mailto:stefan.hanenberg@uni-due.de>) – and do it right from the beginning.

Before you start: do you write your thesis in German or English? In case you write it in German, set the document language to German (Dokument -> Einstellungen -> Sprache) and check whether the generated keywords (content, list of tables, etc. are in german as well as the figure and table name as well).

By the way, this document is written quite quickly and contains probably countless typos. Please do not hesitate to contact me in case you found bugs – the next students (and me) will appreciate your contribution.

Start writing about the context of the work. In case a certain techniques is being studies in this thesis (and it would be rather strange if **not** a certain technique is not being studied in your thesis), in what situations is it applied? How relevant is this technique? Is this technique relevant for teaching, industry or research? If you write for example something about a programming language feature in Java, it makes sense to introduce into programming, the relevance of Java and the language construct itself. When was it introduced and what was the motivation? Typically, you will use a bunch of references in the introduction. For example if you write about a language construct in Java, there are references about programming, Java and the language construct. A general guideline is, that technical references such as URLs should be rather found in footnotes¹.

Since I am the supervisor of you thesis, you probably do a controlled experiment. Hence, you should also have in the introduction why running such an experiment is necessary. From the background perspective, it probably makes sense to write 1-2 paragraphs about why experimentation is necessary at all. You probably will refer to works such as Kaijanaho [4], Ko et al. [6], Vegas et al. [13], or Stefik et al. [12] who state how many experiments can be found in programming language research or software engineering. Do not blindly use those references – check, whether the statements you want make can be justified with these references. And double-check whether you find newer literature (the previous ones are already quite old). You might also want to use some papers that complain about missing empirical studies in the literature (such as [1, 10, 11]).

¹This is an example of a footnote use a <http://myreference.com>

References are written down in the file `bibliography.bib`. By the way: you almost never add a bibtex-entry by hand. Go to webpages such as dblp, Google Scholar, ACM DL, etc. – they give you the bibtex entries that you can just copy (maybe you need to edit the names after copying).

Maybe you even want to make a broader introduction into empirical studies – and even want some philosophical background [8]. In the end, you are relatively free in the introduction. *But keep in mind: the goal is to introduce the reader into the topic. The goal is NOT that you write down everything you know.*

Typically, introductions are quite short – if it requires more than 3-5 pages, there is probably something wrong with it and you should take into account putting some text fragments into other sections. But it is also possible that you decide to put the background and the research question as subchapters in the introduction. It slightly depends on the flow of the text: if the background is rather long, make a separate chapter out of it.

In the beginning of this document you have to fill in:

1. your name,
2. your matriculation number, and
3. your reviewers' names.

Also, write your abstract (in German as well as in English). The abstract is at most half a page – the German as well as the English abstract should fit on one page.

1.1 Research question

This section describes the research question – and it probably takes less than half a page. In order to ease the reading of this Thesis, write at most 1-2 paragraphs in the beginning and then make the research question explicit like this:

RQ: How should you write and structure your thesis?

Maybe you can write down another 1-2 paragraphs here what someone should care about the research question – but not more. By the way: a research question is not a hypothesis – a research question has some more global idea in mind.

1.2 Structure of this work

Write here how the thesis is structured - it takes at most half a page. For the here given document, this chapter could look as follows.

Chapter 2 gives background information relevant for this thesis. Then, chapter 3 introduces into related work. Afterwards, a preliminary experiment is

introduced in chapter 6. Then, the main experiment of this work is presented in chapter 9. Finally, chapter 11 summarizes, discusses and concludes this work.

Note, just because this subchapter looks so boring doesn't mean it is useless: if you are not familiar with this kind of document, it helps you to understand what to find where (and why).

2 Background

The chapter Background gives all information necessary in order to understand this work. If this work is for example on a certain programming language construct, this chapter tells you how this language construct works. Since you also run an experiment, experimentation (or more specific randomized control trials – I assume you construct such an experiment) is a another subchapter. Depending on the kind of further experiments you did (I assume that you also ran some preliminary experiments),

2.1 Your studied technique

Here you introduce into the technique you study. Note, that it is quite probably you will have a number of references here, but that only few of these references (or maybe even none) can be considered as a scientific one. That’s ok, no reason to be ashamed of. It is rather the usual case that techniques are implemented and introduced by industry without any evidence for the benefit of such technique.

```

1 public class HelloWorld
2     public static void main(String[] args) {
3         System.out.println("Hello ,_World");
4     }
5 }
```

Figure 1: Example for “How to write Hello World” in Java - with some smaller font, because otherwise it looks strange

When introducing your technique, make sure that you describe what it actually is, i.e. how it works and give examples of how this technique is applied (see for example Figure 1) . Such example does not only help the reader to understand what the construct is, it also helps you while designing the experiment in what context it is typically applied. **By the way: positioning figures is sometimes a bit annoying. Just use “here if possible” as an option and it should work for you.**

Again, the goal is not to write down everything you know. The goal is to bring the reader into the situation to understand your work – don’t write less than necessary, don’t write more than necessary.

2.2 Randomized control trials

In this section, you probably speak about randomized control trials, or you speak about other kinds of trials, depending on what you do in your work. You should introduce into experimentation in general. Literature about experimen-

tation in software engineering is for example the book by Wohlin et al. [14] or the book by Juristo and Moreno [3], but papers such as the one by Kitchenham et al. might help as well [5].

The goal of this section is give readers all information he needs to understand the experiments you did in in this thesis. It probably makes sense to speak about

- dependent and independent variables,
- experiment layouts, experiment designs, maybe about protocols, and
- analysis methods (which typically means inference statistics) – actually, you need to speak about those things that you are about to explain in this document.

For example, when our ran a crossover trial as part of this thesis, you do not want to write later on in a section “by the way, we did a crossover trial”: then, there is a need to introduce crossover trials in the background chapter. When you did not ran a crossover trial, there is no need to mention it (again – the goal is not that you write down everything you know: the goal is to write a thesis that contains what is needed, but not more, this is a scientific text, not prose).

Introduce into the analysis methods that appear later on. Introduce into the kind of numbers that you need later on (p-values, effect sizes, η_p^2 , etc.). Also, you probably need to introduce into the logic of experimentation, i.e. you probably need to write down what the test for a p-value means and why it has to be compared against some α level, etc.

There might be situations where you cannot argue why certain things are done in a certain way. In the most easist way, it might be unclear for you (for good reasons) why an α level of .05 is used. At a certain point, you can just say “this is a value that is commonly used in the literature”. But now, you should either refer to a paper that gives evidence that this value is commonly used in the literature, or you should enumerate a number of papers that do it that way. **Do not just write down “author X said this is quite common in the literature”.**

You cannot describe everything in detail. For example, when your experiment is analysed using a t-test, give a brief description of the t-test, but then give additional literature where to find more.

I know, it is quite boring to write this stuff down, especially since I handed you already a bunch oh theses that have such a section. It probably feels for you as if you write stuff down that was already written down somewhere else. Please still do it – most readers in software science do not have a background in experimentation and it makes sense to write such things down for them.

3 Related work

In the related work section, you write down what is actually known in the context of your work, i.e. evidence that exists in the context of your work. The goal is not to make any statements whether or not someone believes something is good or not. This section contains evidence. This also implies that your personal opinion does not play any role in this section. **Again, do not state something just because some other author stated it – if you make a statement here, give the corresponding evidence.**

It is hard to estimate how many pages this chapter has, but in case it is less than 3 pages, you probably have not intensively searched for studies that are relevant for your present study.

3.1 Studies on the language construct or similar constructs

In this section you put existing studies that are about the technique you are studying in this thesis. For example, if you are doing a study on the readability of lambda expressions in Java, you mention here those studies that exist about lambdas. Do not describe these studies in a way as “some authors have shown that technique X is beneficial”.

Describe the experiments that have been executed (each one in its own paragraph or subchapter). Describe its experimental design, the way how it has been analyzed, the most important numbers that come from the experiment (p-values, effect sizes) and the interpretation of the experiment. In case there are serious doubts about the validity of the experiment (make sure that these doubts are serious doubts and not just the result of your fantasy), mention them as well.

Of you study for example lambda expressions, the name of subchapter 3.1 will be probably “Studies on lambda expressions”.

3.2 Studies on stuff that influenced your experiment

Again, you describe experimental results here, but from a different perspective: these are studies that have a direct impact on the experiments you did in this thesis. For example, if you generate some code as part of the experiment, you probably need to take into account that identifiers do play a role – so you should check, whether there is evidence on the effect of identifiers on readability (and you will probably describe a number of experiments that were initiated by Dave Binkley such as the one by Lawrie et al. [7]).

In case you study a language construct (which typically means that the choice of the programming language needs plays a role somewhere in this work), it makes sense here to mention studies that show that the choice of the

programming language matters. Although randomized control trials should always be your first choice, it might make sense to take also works as for example the one by Ray et al. [9] into account.

In case you write about a programming language construct, subchapter 3.2 has probably the name “Studies on programming”.

3.3 Studies, Metastudies, etc. that share similarities to yours

There is this other subchapter that play a role. In this chapter, you should mention those experiments that influenced your experimental design. Reasons that you mention those experiments are for example, that a certain measurements is used that you used as well or something else from the experimental design that is necessary for your work. For example, just the decision that you use time to completion (i.e. you let subject do a task until the task has been fulfilled). Note, that it is not necessary to write down such experiments in detail – you just need to mention the experiments and those parts from the experiment that are relevant for you. I.e. this subchapter typically does not require p-values or something.

4 Baseline Indentation Experiment

While researching and developing code snippets for an initial experiment on the readability of nested if-statements, it became apparent that the way indentation length affects reading time was not as straightforward as commonly assumed. This observation motivated the investigation of indentation as a central topic of this thesis. This experiment presents the initial, most basic setup used to study this relationship: nested if-statements with a single-word condition per level, compared across four fixed indentation lengths.

4.1 Initial Considerations

We want to design a controlled experiment where the participant receives code snippets of nested if-statements, in order to measure the effect of different indentation lengths and nesting levels. To compare the different lengths and levels, we want to measure the reading time for each code snippet. For this purpose, we measure the reaction time, i.e., the time it takes for the participant to complete the task, which serves as an approximation for the reading time.

Programming Language: The experiment presents JavaScript code snippets to the participants, as the syntax for nested if-statements is common in this language.

Indentation Length: This experiment aims to determine whether higher indentation lengths result in faster reaction times than lower indentation lengths.

Nesting Level: The scope of this experiment is to measure which nesting levels require the most time.

Code Snippets: Each condition consists of a single word, randomly chosen from a noun list.

Highlighted Condition: In each code snippet, one condition is highlighted with a red background, indicating that this specific condition fails. This causes the corresponding if-statement, along with all subsequent nested if-statements, to not be executed.

Task: The task is to determine which else-branch is executed as a result of the highlighted condition failing. Since the highlighted condition determines the nesting level at which execution stops, the participant has to identify the corresponding numeric output associated with that level and press "1" once they have found it.

In Between Breaks: After pressing "1", a new window appeared displaying the result of the previous task along with a prompt inviting the participant to take a break if needed. This allowed the participant to verify that the previously identified output was correct, and to rest between tasks in case of fatigue or loss of concentration, before proceeding to the next code snippet by pressing "Enter".

Training: There is a training phase before the actual experiment starts to reduce learn effects.

4.2 Experiment Layout

The experiment is designed as a crossover N-of-1 experiment with only one participant. The participant receives randomly generated code snippets combining the following parameters:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his results. This variable should approximate the reading time.
- **Independent Variables:**
 - **Indentation_Length:** The indentation lengths used for this task. For this variable there are four different treatments: 2, 4, 6, and 8.
 - **Nesting_Level:** The nesting-levels used for this task. For this variable there are nine different treatments: 1, 2, 3, 4, 5, 6, 7, 8 and 9.
- **Fixed Variables:**
 - **repetitions:** The experiment repeats each treatment combination 5 times for a total of 180 code snippets.
- **Task:** *"Your task is to identify, where the closing bracket of the corresponding if-statement appears. Each bracket has a corresponding number. [...] press [1] when you found it"*

The goal of the Experiment is to see if the independent variables influence the dependent variables.

4.3 Experiment Design

Each JavaScript code snippet consists of a chain of nine nested if-statements, generated recursively so that each if-statement's then-branch is either the next nested if-statement or, at the innermost level, a return value of 0. Each if-statement's else-branch returns a distinct integer between 1 and 10, these ten

values then are randomly shuffled and assigned across the nine nesting levels, ensuring that the correct answer cannot be assigned from the position of the highlighted condition alone. Each nested if-statement is printed on its own line, indented by a fixed number of spaces equal to the product of the indentation length treatment and the nesting depth. The condition corresponding to the treatment's Level is highlighted with a red background, indicating that this if-statement, and all deeper nested if-statements, evaluate to false, so that the associated else-branch's return value is the correct answer. Figure 2 shows an example code snippet.

```

if(example) {
  if(recommendation) {
    if(fly) {
      if(straw) {
        if (pocket) {
          if(ox) {
            if(cherry) {
              if(warehouse) {
                if(limitation) {
                  //0
                } 7
              } 1
            } 2
          } 4
        } 5
      } 8
    } 9
  } 6
} 10

```

Figure 2: Example code snippet with nesting depth 9, indentation length 2 and the highlighted condition marks the target if-statement on nesting level 5.

4.4 Analysis

The measured reaction times were analyzed using an ANOVA. Table 1 shows that both Indentation_Length ($p < .001$, $\eta_p^2 = .670$) and Nesting_Level ($p < .001$, $\eta_p^2 = .469$) had a large and statistically significant effect on reaction time, with a significant interaction between the two ($p < .001$, $\eta_p^2 = .575$).

The effect of indentation length was substantial: an indentation length of 2 was, on average, 94.3% slower than a length of 8, and still 45.8% slower than a length of 6. This gap narrowed considerably between the longer lengths, with length 6 being only 9.6% slower than length 8. This suggests that the benefit

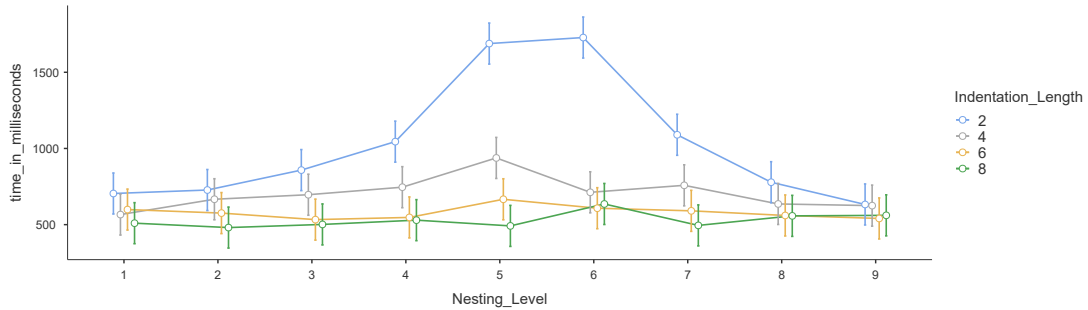
Table 1: Experiment results for the Baseline Indentation Experiment

Variable	df	F	p	η_p^2	TRT	N	$CI_{95\%}$		M
							Lower	Upper	
Indentation_Length	3	97.33	<.001***	.670	2	45	983	1073	1028
					4	45	660	750	705
					6	45	535	625	580
					8	45	484	574	529
Nesting_Level	8	15.92	<.001***	.469	1	20	527	662	595
					2	20	545	680	613
					3	20	580	715	647
					4	20	649	784	717
					5	20	879	1013	946
					6	20	854	988	921
					7	20	666	801	733
					8	20	566	700	633
					9	20	522	657	590
Indentation_Length $\times$ Nesting_Level	24	8.12	<.001***	.575	–	–	–	–	–

of increasing indentation length diminishes quickly, with most of the improvement occurring between very short and moderate lengths, rather than continuing at the same rate for longer lengths.

The estimated marginal means for Nesting_Level showed a pronounced inverted U-shaped pattern, with the slowest reaction times occurring around the center of the nested structure, level 5 was 60.3% slower than level 1 and 60.3% slower than level 9. Levels 6 and 7 were similarly slow, while the outermost levels on both ends were read comparatively quickly.

The interaction between Indentation_Length and Nesting_Level (see Figure 3) shows that this U-shaped pattern was almost entirely driven by the shortest indentation length. At a length of 2, reaction times varied dramatically across nesting levels, from 632 ms at level 9 to 1728 ms at level 6, a difference of over 173%. At a length of 8 this variation nearly disappeared with a difference of just 32%. In other words, the strong dependency of reading time on nesting level observed for short indentation lengths becomes far less pronounced, though not entirely absent, as indentation length increases.

Figure 3: Interaction diagram Indentation_Length $\times$ Nesting_Level

4.5 Discussion

This experiment shows that a higher `Indentation_Length` was, in this setup, consistently associated with faster reaction times than the previous, shorter length. While reaction times did decrease substantially as indentation length increased, the size of this improvement was highly uneven. The pattern resembles a diminishing return: the difference between a length of 2 and a length of 4 was far larger than the difference between a length of 6 and a length of 8, suggesting that the benefit of additional indentation falls off rather than continuing linearly.

The strong interaction between indentation length and nesting level indicates that indentation length does not affect all nesting levels equally. For a short indentation length, conditions near the center of the nested structure took over 170% longer to process than conditions at the outer levels, whereas this discrepancy was much smaller for longer indentation lengths. This suggests that some structural property of the code, related to a condition's position within the nesting hierarchy, interacts with indentation length in a way that no single indentation length could uniformly account for. This experiment therefore represents the empirical starting point of this thesis. Why nesting level and indentation length interact so strongly is a question that could not yet be fully answered at this stage.

5 Commonalities between Preliminary Experiments

Multiple preliminary experiments were created with the common goal of ultimately building a main experiment comparing the readability of indentation styles in nested if-statements. These preliminary experiments share commonalities in terms of initial considerations, experiment design, and conduction. These commonalities are described in this section, while the following chapters focus on the aspects specific to each experiment.

5.1 Initial Considerations

Each experiment presents the participant with code snippets of nested if-statements, in which one condition is highlighted to indicate that it fails. To compare the different treatments, we want to measure the reading time for each code snippet. For this purpose, we measure the reaction time, i.e., the time it takes for the participant to complete the task, which serves as an approximation for the reading time.

Programming Language: The experiments present JavaScript code snippets to the participants, as the syntax for nested if-statements is common in this language.

Highlighted Condition: In each code snippet, one condition is highlighted with a red background, indicating that this specific condition fails. This causes the corresponding if-statement, along with all subsequent nested if-statements, to not be executed.

Task: The task is to determine which else-branch is executed as a result of the highlighted condition failing. Since the highlighted condition determines the nesting level at which execution stops, the participant has to identify the corresponding numeric output associated with that level and press "1" once they have found it.

N-of-1: All preliminary experiments were conducted as crossover N-of-1 trials with the author as the only participant. N-of-1 trials give the participant all treatment combinations, making it possible to study the effect of the independent variables on a single participant.

In Between Breaks: After pressing "1", a new window appeared displaying the result of the previous task along with a prompt inviting the participant to take a break if needed. This allowed the participant to verify that the previously identified output was correct, and to rest between tasks in case of fatigue or loss of concentration, before proceeding to the next code snippet by pressing "Enter".

Random Position Start: To avoid the participant's eyes always starting at the same position on the screen when a new code snippet appeared, the position at which the code snippet was displayed was randomized for each task. This prevents participants from developing a fixed viewing pattern that could artificially reduce reading time independent of the actual treatment.

Training: There is a training phase before the actual experiment starts to reduce learn effects.

5.2 Measurement and Conduction

The measurement was conducted using an interactive interface that generated and presented the code snippets as plain text. The participant completed each task with a key press, and the reaction time in milliseconds was measured from the moment the code snippet appeared until the participant pressed a key. After the experiment, a csv-file was generated containing all measured results.

6 Chained Conditions Preliminary Experiment

In practice conditional statements can chain multiple boolean expressions together using `&&`, which increases the horizontal length of each condition. This preliminary experiment investigates how chained conditions affect the readability of nested if-statements, and whether the previously observed relationship between indentation length and reading time still holds when the last condition is highlighted and conditions themselves vary in length.

6.1 Initial Considerations

We want to design a controlled experiment where the participant receives code snippets of nested if-statements with chained boolean conditions, in which the last condition fails and measuring the effect of different indentation lengths and nesting levels. To compare the different lengths and levels we want to measure the reading time for each code snippet, which is done by measuring the reaction time, i.e., the time it takes for the participant to complete the task, which serves as an approximation for the reading time. See further considerations common to all preliminary experiments in Section 5.

Indentation Length This experiment aims to determine whether higher indentation lengths consistently result in faster reaction times than lower indentation lengths, or whether this relationship changes once conditions are chained and vary in length.

Nesting Level The scope of this experiment is to measure which nesting levels require the most time.

Chained Condition To examine the effect of longer conditions on readability in nested if-statements we decided to use random words that are chained as boolean conditions. Chaining four boolean expressions with `&&` increase the horizontal length of a condition.

Highlighted Condition In each code snippet the last condition of one chained boolean expression is highlighted with a red background, this conditions fails. This causes all nested if-statements afterwards not to be executed. Because the last rather than the first condition is highlighted, the participant cannot determine the result by simply reading straight down the code, but must instead scan the chained condition to locate the beginning of the row.

6.2 Experiment Layout

As previously discussed, the experiment is designed as a crossover N-of-1 experiment with only one participant. The participant receives 225 randomly gen-

erated code snippets which all contain these different parameters:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his results. This variable should approximate the reading time.
- **Independent Variables:**
 - **indentation length:** The indentation lengths used for this task. For this variable there are five different treatments: 2, 4, 5, 6 and 8,
 - **Nesting_Level:** The nesting-levels used for this task. For this variable there are nine different treatments: 1, 2, 3, 4, 5, 6, 7, 8 and 9.
- **Fixed Variables:**
 - **repetitions:** The experiment repeats each treatment combination 5 times for a total of 225 code snippets.
- **Task:** *“Your task is to identify, where the closing bracket of the corresponding if-statement appears. Each bracket has a corresponding number. [...] press [1] when you found it”*

The goal of the Experiment is to see if the independent variables influence the dependent variables.

6.3 Experiment Design

Each JavaScript code snippet, will randomly appear on the monitor, but will not reach out of the monitor so the participant can always see the whole code without scrolling, consists of a chain of nine nested if-statements, with else-branch return values (1 to 10) randomly shuffled across nesting levels. In this experiment, each if-statement’s condition consists of a chain of four words joined by &&, randomly chosen without repetition from a predefined noun list. The last word of the chain at the nesting level corresponding to the treatment’s Level is highlighted with a red background, indicating that the overall condition fails and that the associated else-branch’s return value is the correct answer. Figure 4 shows an example code snippet.

6.4 Analysis

The measured reaction times were analyzed using an ANOVA. Table 2 shows that both Indentation_Length ($p < .001$, $\eta_p^2 = .128$) and Nesting_Level ($p < .001$, $\eta_p^2 = .604$) had a statistically significant effect on reaction time, while their interaction did not reach significance ($p = .140$, $\eta_p^2 = .189$). The indentation length of 2 was on average 11.9% slower than a length of 8 and still 8.5% slower than the

```

if(stone && alarm && thread && projection) {
  if(peace && railway && fly && leadership) {
    if(dogs && turn && birds && test) {
      if(navigation && pin && war && plate) {
        if(giraffe && word && song && position) {
          if(son && division && tub && infrastructure) {
            if(shape && bells && reinforcement && car) {
              if(fowl && need && drop && laboratory) {
                if(page && space && on && vein) {

                } 5
              } 7
            } 6
          } 9
        } 10
      } 8
    } 3
  } 4
} 2

```

Figure 4: Example code snippet with nesting depth 9, indentation length 2 and the highlighted condition marks the target if-statement on nesting level 7.

length of 4. Between length 4 and 8 the gap narrowed with length 4 being only 3.1% slower. This suggests that most of the benefit of increasing indentation length was already realized by length 4, with lower gains beyond that point. Length 6 is 3.7% slower than length 4.

The estimated marginal means for *Nesting_Level* again showed a pronounced inverted U-shaped pattern, with the slowest reaction times occurring around the center of the nested structure. level 5 was 34.6% slower than level 1 and 50.8% slower than level 9. Levels 4 and 6 were similarly slow. Because the interaction between *Indentation_Length* and *Nesting_Level* was not significant, this U-shaped pattern appears to hold consistently across indentation lengths, rather than being driven disproportionately by any single one like before (see Section 3).

6.5 Discussion

The results replicate the general pattern from the baseline experiment (See Section 4.5) but providing important insights, because increasing the indentation length does not straightforwardly translate into faster reading times. While length 8 was again the fastest condition and length 2 the slowest, the intermediate lengths did not decrease monotonically. Length 6 was slower than length

Table 2: Experiment results for the Indentation Length by Nesting Level Experiment

ANOVA on Reaction Time (approximator for Reading Time)									
Variable	df	F	p	η_p^2	TRT	N	$CI_{95\%}$		M
							Lower	Upper	
Indentation_Length	4	6.63	<.001***	.128	2	45	804	855	830
					4	45	739	790	765
					5	45	738	789	763
					6	45	765	817	791
					8	45	716	768	742
Nesting_Level	8	34.26	<.001***	.604	1	25	635	704	670
					2	25	715	784	749
					3	25	820	890	855
					4	25	833	903	868
					5	25	867	936	902
					6	25	822	892	857
					7	25	752	821	787
					8	25	683	752	717
					9	25	563	632	598
Indentation_Length $\times$ Nesting_Level	32	1.31	0.140	.189	–	–	–	–	–

4. This breaks the assumption that “more indentation is always better”. One explanation could be that once a moderate indentation length is reached, further increase may impose their own costs. For example, wider horizontal spacing could require more eye movement per line or push code closer to the edge of comfortable visual scanning and the participant could need more time. But Length 8 is still faster than 6. Another explanation could be that some incidental properties of the randomly generated conditions, such as word length or visual homogeneity, could have played a role. What can be said is that a higher indentation length as a standalone factor is not always beneficial, but it is important to be said that the current design, given the moderate sample size ($N = 45$ per condition), cannot fully disentangle this effect from measurement noise.

7 Dynamic Indentation Preliminary Experiment

The previous experiment showed that indentation length can affect reading time in a non-linear way, with a notable deviation at an indentation length of six. This raises the question of whether a fixed indentation length is well suited to represent the structure of nested if-statements at all. This preliminary experiment introduces dynamic indentation, a indentation style where each nested if-statement is indented just far enough that it begins directly one empty space after the opening brace of the preceding condition. Unlike fixed indentation lengths, dynamic indentation adapts to the length of each condition individually. This experiment investigates if reading time is affected by the length of the words used in the conditions and by the nesting level, when dynamic indentation is used instead of a fixed indentation length.

7.1 Initial Considerations

We want to design a controlled experiment where the participant receives code snippets of nested if-statements with dynamic indentation. To compare the different condition word lengths and levels we want to measure the reading time for each code snippet, which is done by measuring the reaction time, i.e., the time it takes for the participant to complete the task, which serves as an approximation for the reading time. See further considerations common to all preliminary experiments in Section 5.

Indentation Length This experiment exclusively uses dynamic indentation, an indentation style in which each nested if-statement is indented just far enough to begin directly one space after the opening brace of the preceding condition.

Nesting Level The scope of this experiment is to measure which nesting levels require the most time.

Word Length To examine the effect of different word lengths used in the conditions, the words in the if-statements are chosen from a range of the target length up to two characters longer.

7.2 Experiment Layout

As previously discussed, the experiment is designed as a crossover N-of-1 experiment with only one participant. The participant receives 180 randomly generated code snippets which all contain these different parameters:

- **Dependent Variables:**

- **reaction time:** The time it takes the participant to input his results. This variable should approximate the reading time.

- **Independent Variables:**

- **Condition_Length** The target condition word length used for this task. For this variable there are four different treatments: two, four, six, and eight characters. Note that the actual word length may exceed the target by up to two characters, as words are drawn from a range of the target length up to two characters longer.
- **Nesting_Level:** The nesting-levels used for this task. For this variable there are nine different treatments: one, two, three, four, five, six, seven, eight and nine.

- **Fixed Variables:**

- **repetitions:** The experiment repeats each treatment combination 5 times for a total of 180 code snippets.
- **Task:** *“Your task is to identify, where the closing bracket of the corresponding if-statement appears. Each bracket has a corresponding number. [...] press [1] when you found it”*

The goal of the Experiment is to see if the independent variables influence the dependent variables.

7.3 Experiment Design

```

if (as) {
    if (no) {
        if (we) {
            if (us) {
                [...]
            } 2
        } 5
    } 4
} 3

```

Figure 5: Shortened example code snippet with dynamic indentation. The highlighted condition is on nesting level 1 with a condition length of 2.

Each JavaScript code snippet, will randomly appear on the monitor, but will not reach out of the monitor so the participant can always see the whole code

without scrolling, consists of a chain of nine nested if-statements, with else-branch return values (1 to 10) randomly shuffled across nesting levels. Each if-statement's condition consists of a single word, randomly chosen from a pre-defined noun list (or, for the shortest condition length, from a fixed list of two-letter words) to match the treatment's target condition length within a tolerance of two characters. Dynamic indentation was implemented using a continuation-style rendering where each nested if-statement is printed directly after the single space following the opening brace of the preceding if-statement, rather than on a new line with fixed indentation. This causes the indentation of each level to depend on the horizontal length of the preceding condition, producing a staircase-like layout. Figure 5 shows a shortened example code snippet with dynamic indentation.

7.4 Analysis

Table 3: Experiment results for the Nesting Level by Condition Word Length Experiment

ANOVA on Reaction Time (approximator for Reading Time)									
Variable	df	F	p	η_p^2	TRT	N	$CI_{95\%}$		M
							Lower	Upper	
Nesting_Level	8	1.429	0.189	.074	1	20	399	427	413
					2	20	386	415	401
					3	20	404	432	418
					4	20	383	412	398
					5	20	388	416	402
					6	20	401	430	415
					7	20	378	407	393
					8	20	386	414	400
					9	20	388	417	402
Condition_Word_Length	3	1.081	0.359	.022	2	45	399	418	409
					4	45	391	410	400
					6	45	400	419	409
					8	45	391	410	401
Nesting_Level $\times$ Condition_Word_Length	24	0.958	0.524	.138	–	–	–	–	–

The measured reaction times were analyzed using an ANOVA. Table 3 shows that neither Nesting_Level ($p = .189$, $\eta_p^2 = .074$) nor Condition_Word_Length ($p = .359$, $\eta_p^2 = .022$) had a statistically significant effect on reaction time, and their interaction also did not reach significance ($p = .524$, $\eta_p^2 = .138$). The estimated marginal means for Nesting_Level fluctuated only slightly across the nine levels, with the slowest level (3) being roughly 6% slower than the fastest level (7). Level 1 was only marginally slower than level 3, and the remaining levels clustered closely around the overall mean. Reaction times no longer varied systematically depending on a condition's position within the nesting hierarchy.

7.5 Discussion

Unlike the two preceding experiments on fixed indentation, this experiment found no significant effect of either nesting level or condition word length on reaction time when dynamic indentation was used. This stands in contrast to the baseline experiment (see Section 4.5, and chained conditions experiment (see Section 6.5), where nesting level showed a U-shaped pattern. This suggests that dynamic indentation may eliminate, or at least substantially reduce, the influence that nesting level had under fixed indentation, where reaction times followed a pronounced inverted U-shaped pattern depending on the position of the highlighted condition. One possible explanation is that dynamic indentation provides a more direct visual path to each condition, regardless of its nesting level, whereas fixed indentation may have caused participants to expend additional effort locating conditions in the middle of the nested structure. Notably, the overall reaction times measured in this experiment (approximately 380–430 ms) were considerably lower than those observed in the previous experiment on fixed indentation (approximately 560–950 ms). While this experiment was not designed as a direct comparison between fixed and dynamic indentation, this difference may indicate that dynamic indentation allows for faster comprehension overall, independent of nesting level or word length. However, as no statistical test was conducted between the two experiments, this observation should be interpreted with caution and would need to be confirmed in a dedicated follow-up experiment directly comparing both indentation styles under identical conditions. The absence of a significant effect for condition word length is consistent with the expectation that dynamic indentation, by construction, compensates for varying condition lengths: since each nested if-statement is positioned directly after the preceding condition's opening brace, differences in word length are absorbed into the layout itself rather than affecting the reader's ability to locate the relevant condition. These findings support the idea that dynamic indentation may be a promising alternative to fixed indentation lengths, warranting further investigation in a experiment that directly compares both approaches under controlled conditions.

8 Overlap Preliminary Experiment

While designing the previous experiments, it became apparent that long conditions can visually overlap with the conditions of adjacent nesting levels when a fixed indentation length is used. This overlap was not systematically controlled nor its effect on reading time was measured. This preliminary experiment addresses this issue by manipulating the amount of visual overlap between a highlighted condition and its neighboring conditions in order to investigate whether overlap itself, rather than the condition length or indentation length, affects reading time. Since dynamic indentation positions each nested if-statement directly after the preceding condition, it inherently prevents such overlap from occurring. This experiment will also compare dynamic indentation to fixed indentation lengths under conditions specifically designed to produce high overlap, to examine whether dynamic indentation can mitigate this effect.

8.1 Initial Considerations

We want to design a controlled experiment where the participant receives code snippets of nested if-statements, in which the amount of visual overlap between the highlighted condition and its neighboring conditions is controlled. To compare the effect of overlap across different indentation styles, we want to measure the reading time for each code snippet. For this purpose, we measure the reaction time, i.e., the time it takes for the participant to complete the task, which serves as an approximation for the reading time. See further considerations common to all preliminary experiments in Section 5.

Indentation Length: This experiment compares three fixed indentation lengths (6, 10 and 14) as relatively high indentation lengths and dynamic indentation, allowing us to examine whether the effect of overlap differs depending on the indentation style used.

Overlap: To manipulate the amount of visual overlap independently of indentation style, two treatments are used: Overlap and No_Overlap. In the Overlap condition, the word lengths of the highlighted condition and its neighboring conditions, one nesting_level above and below, were chosen to produce a substantial overlap for a fixed indentation length, with these same word lengths reused across all indentation styles. As a result the overlap varies with indentation length, but is always present and relatively higher on lower indentation lengths. The No_Overlap condition has shorter word lengths instead and no overlap occurs and these shorter word lengths are likewise reused across all indentation styles. In dynamic indentation, no overlap can

occur, because each nested if-statement directly starts after the preceding condition's opening brace, but the same word length baselines are still used to keep the underlying code snippets comparable across indentation styles.

8.2 Experiment Layout

As previously discussed, the experiment is designed as a crossover N-of-1 experiment with only one participant. The participant receives randomly generated code snippets combining the following parameters:

- **Dependent Variables:**
 - **reaction time:** The time it takes the participant to input his result. This variable should approximate the reading time.
- **Independent Variables:**
 - **Indentation_length:** The indentation length used for this task. For this variable there are four different treatments: fixed indentation lengths of 6, 10 and 14, as well as dynamic indentation.
 - **Overlap:** Whether the code snippet was generated using word length that produce overlap for fixed indentation length. For this variable there are two treatments: Overlap and No_Overlap
 - **Nesting_level:** The nesting_level of the highlighted condition. For this variable there are five different treatments: 2, 3, 4, 5 and 6, because overlap can not be above Nesting_level 1 or below Nesting_level 7.
- **Fixed Variables:**
 - **repetitions:** The experiment repeats each treatment combination 10 times for a total of 400 code snippets.
- **Task:** *"Your task is to identify, where the closing bracket of the corresponding if-statement appears. Each bracket has a corresponding number. [...] press [1] when you found it"*

The goal of the experiment is to see if the independent variables influence the dependent variables.

8.3 Experiment Design

In the experiment the participant is shown a Javascript code snippet with seven nested if-statements, which randomly appears on the monitor, but will not reach out of the monitor so the participant can always see the whole code without scrolling. Each snippet contains a combination of the values for independent

```

if(reasonbirt) {
    if(waxsnakecarpentermyunitbedleadershipr) {
        if (professionperseveranceroutecampapprox) {
            if(archshadesubstancegolaceaxyamounta) {
                if(considerat) {
                    if(bridgetopt) {
                        if(jellyfisha) {

                        } 7
                    } 2
                } 5
            } 3
        } 4
    } 1
} 6

```

Figure 6: Example code snippet in the Noise condition with a indentation length of 6. The highlighted condition is located at nesting level 3 and visually overlaps with the longer words at the neighboring nesting levels above and below.

variables. For most nesting levels, a condition word of moderate, fixed length is used, so that these levels do not interfere with the measurement of overlap at the highlighted condition. The word lengths of the highlighted condition and its two neighboring conditions, one nesting level above and one level below, are instead set separately, depending on the Overlap treatment. As in the previous experiments, the return values assigned to the else-branches are randomly shuffled across nesting level. Condition words of a given target length were constructed by concatenating randomly drawn nouns from a predefined noun list until the combined string reached or exceeded the target length, after which the result was truncated to exactly that length. This procedure was necessary because natural nouns of the required length, particularly for the long words used in the Overlap condition, were not reliably available in the noun list; concatenating multiple nouns made it possible to generate condition words of arbitrary length while still being composed of recognizable word fragments rather than random character strings.

It is important to differentiate between the overlap as a manipulation of the underlying word lengths and the actual visual overlap that results once a code snippet is rendered. The Overlap and No_Overlap treatments determine the word length used for the highlighted condition and its neighbors, independently of indentation style. In the Overlap condition, these three words are chosen to be much longer than the words used at the other levels, causing their text to extend into the horizontal space of the adjacent lines when a fixed indentation length is used. In the No_Overlap condition, these three words are

instead chosen to be short, matching the length used for the other levels, so that no overlap occurs regardless of indentation length. For fixed indentation lengths, this word-length manipulation reliably produces visual overlap in the Overlap condition. For dynamic indentation, code snippets are generated and presented to the participant using the same Overlap and No_Overlap word-length manipulation as for fixed indentation lengths, even though no visual overlap can occur in either case, since dynamic indentation always positions each nested if-statement directly after the preceding condition’s opening brace. This allows the effect of indentation style to be isolated while holding the underlying code content constant, and makes it possible to distinguish whether an observed effect is caused by the visual overlap itself, or merely by the increased word length used to produce it. See Figure 6 for an example code snippet.

8.4 Analysis

Table 4: Experiment results for the Indentation Length by Overlap by Nesting Level Experiment

ANOVA on Reaction Time (approximator for Reading Time)									
Variable	df	F	p	η_p^2	TRT	N	$CI_{95\%}$		M
							Lower	Upper	
Indentation_Length	3	39.45	<.001***	.247	6	100	502	524	513
					10	100	445	467	456
					14	100	431	454	443
					Dynamic	100	422	444	433
Overlap	1	82.95	<.001***	.187	No_Overlap	200	427	443	435
					Overlap	200	479	495	487
Nesting_Level	4	11.54	<.001***	.114	2	80	482	507	494
					3	80	456	481	469
					4	80	446	471	458
					5	80	432	457	444
					7	80	428	453	440
Indentation_Length $\times$ Overlap		3	9.84	<.001***	.076	–	–	–	–
Indentation_Length $\times$ Nesting_Level		12	1.63	0.081	.052	–	–	–	–
Overlap $\times$ Nesting_Level		4	2.68	0.032*	.029	–	–	–	–
Indentation_Length $\times$ Overlap $\times$ Nesting_Level		12	1.51	0.119	.048	–	–	–	–

The measured reaction times were analyzed using an ANOVA, Table 4 shows that Indentation_Length, Overlap and Nesting_Level all had statistically significant effect on reaction time($p < .001$ for all three). Overall, dynamic indentation resulted in the fastest average reaction time. It was 15.6% faster than an indentation length of 6, though the advantage over longer fixed lengths was far smaller: only 5.0% faster than length of 10, and 2.3% faster than length of 14. Overlapping conditions were read about 12% slower than non-overlapping ones, averaged across indentation styles. The interaction between Indentation_Length and Overlap shows that this overlap penalty is not evenly distributed across indentation styles, but decreases as the fixed indentation length increases: at a length of 6, overlapping conditions took 17.8% longer than non-overlapping ones and this penalty gets lower to 16.1% at length 10 and 13.3% at length 14. For dynamic indentation, the penalty was negligible at only 0.5%, ef-

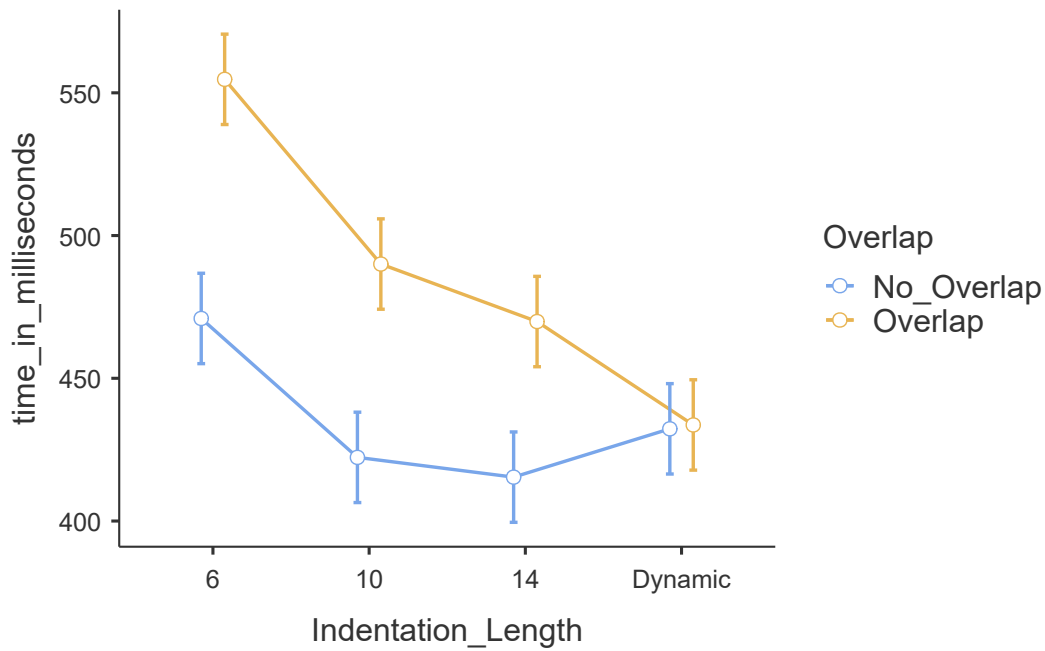


Figure 7: Interaction diagram Indentation_Length $\times$ Overlap

fectively eliminating the effect of overlap seen at every fixed indentation length.

This ranking shifts, depending if overlap is present or not. Without overlap, dynamic indentation was not the fastest style, it ranked third, behind length 14 and length 10, it was only clearly faster than length 6. Once overlap was introduced, however, this ranking reversed entirely: dynamic indentation became the fastest style overall, ahead of length 14, length 10, and length 6, in that order. In other words, dynamic indentation only overtakes fixed indentation once conditions are long enough to cause overlap, but never performs substantially worse even when they are not.

8.5 Discussion

The results show that both indentation length and the presence of overlap affect the reading time. Also these two factors interact strongly. Indentation lengths benefit from short non-overlapping conditions, but shorter indentation lengths means longer reaction time because conditions became long enough to visually overlap. Dynamic indentation, by contrast, showed almost no difference between the Overlap and No_Overlap treatments (a difference of only 0.5%). This is a central result of this experiment, but it requires some clarification regarding what "Overlap" means in the context of dynamic indentation. Since dynamic indentation always positions each nested if-statement directly after the preceding condition's opening brace, no visual overlap can occur under this indentation style, regardless of how long the condition words are. The Overlap

treatment was nonetheless applied to dynamic indentation as well, using the same long word lengths as for the fixed indentation lengths, even though these words could not produce any actual visual overlap. In this sense, the "overlap" tested under dynamic indentation is not a true visual overlap, but rather a word-length manipulation applied without its usual visual consequence.

This design choice was deliberate, and it is precisely what makes the comparison meaningful: it allows the word length used in the Overlap condition to be held constant across all indentation styles, isolating the effect of visual overlap itself from the mere effect of longer words. If the slowdown observed for fixed indentation lengths in the Overlap condition had been caused simply by the increased word length, one would expect a comparable slowdown for dynamic indentation as well, since the same long words were used there too. Instead, dynamic indentation showed virtually no difference between long and short condition words, while every fixed indentation length showed a substantial one. This indicates that the slowdown observed for fixed indentation lengths is not primarily driven by word length itself, but specifically by the visual overlap that long words produce once a fixed indentation step is used. Overlap, rather than word length in general, therefore appears to be the more direct cause of the increased reading time observed here.

This interpretation is further supported by a shift in the ranking of indentation styles depending on the presence of overlap. Without overlap, dynamic indentation was not the fastest option; both length 10 and length 14 outperformed it slightly, and only length 6 was clearly slower. Once overlap was introduced, however, this ranking reversed completely: dynamic indentation became the fastest style overall, ahead of every fixed indentation length, including 14. This shift illustrates the core trade-off between fixed and dynamic indentation: fixed indentation lengths can be marginally faster under favorable conditions, when condition words happen to be short, but they carry the risk of a substantial penalty whenever conditions turn out to be long. Dynamic indentation sacrifices little of this best-case performance while entirely avoiding the worst case, making it the more robust choice when the length of conditions in real code cannot be known or controlled in advance.

Taken together, these findings support the central motivation of this thesis: that dynamic indentation is not simply a stylistic alternative to fixed indentation, but a structural solution to a specific readability problem, namely the visual overlap that can arise when condition length and indentation step size are mismatched. As this experiment was conducted with a single participant, these results should be interpreted as preliminary. They motivate a main experiment involving multiple participants, in which the robustness of dynamic indentation against varying condition lengths can be evaluated more.

9 Main Experiment

So, here your main experiment is written. What is quite typical is, that this chapter has hardly any references – in case references are necessary in this section, you should take into account to write them down in section 3.

Note that you do not write about an experiment that is executed up to

9.1 Initial considerations

The initial considers are a section that is quite important. It introduces those questions that need to be considered in the experiment design. For example, when you study a certain programming language construct and such construct is implemented by multiple languages, the choice of the language is an initial consideration. Structure such considerations in a way that one can follow what needs to be taken into account in the experimental design.

Still, you probably start this section with a description of a rough idea what the goal of the experiment is – something that is slightly more specific than the research question (see section 1.1): such idea is necessary in order to understand what the initial considerations are relevant.

- **Programming Language:** Since language construct X is available in multiple programming languages and the choice of the programming language has an effect on the expected results (see chapter 3.2), a programming language needs to be chosen that is appropriate for the experiment. Since the experiment does not plan an exhaustive training session, it is desirable to use a programming language that the assumed participants are able to handle.
- **Programming Environment:** It can be assumed that the programming environment (such as IDEs, etc.) play a role for software development (see chapter 3.2).
- **Programming Tasks:** Tasks need to be given to the subjects. The goal is to find tasks that are
- **Measurements technique:**
-

Note that the goal of this subchapter is to guide the reader through the questions that need to be addressed by the experimental design. This subchapter does not provide answers or decisions. It provides questions that need to be addressed.

It is not unusual that this subchapter has a number of pages – if it has less than 1.5 pages, there is probably something wrong.

9.2 Hypotheses

Ok, now it is time for the the hypotheses. Maybe you just have one single hypothesis, maybe a bunch of them. You should write down the null-hypothesis and often you will not write down the alternative hypothesis (which is just a negation of the hypothesis).

H₀: Having a thesis template such as the present one does not have an influence on the final mark in comparison to theses written without such a template.

Keep in mind that a null hypothesis say that “there is no difference” – since you are doing an experiment, you should be familiar with it. Make sure that a hypothesis is something that can be rejected by observations, i.e. a hypothesis it not as vague as the research question (see section 1.1): it is possible for someone who did an observation (i.e. who ran an experiment) to decide whether or not H₀ should be rejected.

In case you have multiple hypotheses, write them down in a way that it is understandable for the reader later on to what hypothesis you refer to. For example, it could look like the following:

H_{0_{finalMark}}: Having a thesis template such as the present one does not influence the final mark in comparison to theses written without such a template.

H_{0_{time}}: Having a thesis template such as the present one does not influence the time required to write a thesis in comparison to theses written without such a template.

In case you are running an experiment with multiple factors, do not write down all hypotheses (all combinations of factors), but concentrate on those ones that are essential for this work (main factors).

This subchapter probably takes just half a page.

9.3 Measurement technique

Here you describe how and what you will measure in your experiment. You have already described in the section 9.1 what questions concerning the measurements techniques are. For example, the questions could be that you have a bunch of alternatives such as reaction time, time to completion, etc. Here you say, how exactly you measure. This section should be readable for someone who wants to redo your experiment (without knowing your experiment's results).

The measurements technique could be a tool that you have written in order to measure your subjects (actually, this is most often the case). In such a situation you introduce this tool here – and it helps a lot if you also add a screenshot.

It cannot be state upfront how much space is required here:

- In case you do a simple experiment where you give subjects an empty sheed and start the stop watch (that you stop when the subjects say they have finished), then this section is probably just half a page long.
- In case you write a complicated tool where a lot of interactions are possible for the subjects, it takes probably a number of pages.

9.4 Experimental design

In some situations it makes sense to swap the chapter 9.4 and 5, especially if it is not yet clear what the different parts of the experiment will be.

In the section experimental design, you need to point of how the experiment looks like. Note, that the chapter experimental layout exists as well, i.e. there is a section where you make explicit

9.4.1 Teaching and Teaching Material

In case you need to teach your subjects, define here how the subjects should be taught. Define the expectations and the material for the subjects. There is no need to give here the raw materials you have to the subjects – but add them to the appendix (in case the resulting number of pages is acceptable), otherwise add them to an external storage that you add to this thesis.

9.4.2 Tasks

This is probably the most important and interesting part of this section. Here you describe, what tasks you give to the subjects. In chapter 9.1 you already described what kinds of tasks could be taken into account. Here, you define the concrete tasks you give to the subjects.

It is easily possible that this subchapter alone takes more than one page (depending on the complexity and the number of tasks, it is possible that this subchapter has 5-10 pages – in such a case, take into account to make a chapter out of it).

Tell precisely what tasks will be done and give some examples of the tasks here – and add all tasks to the appendix (just in case the tasks require countless

9.5 Experimental layout

This section is relatively short – probably just half a page, maybe one page. It tells you about the formal model underlying the experiment. You should start with the description of the dependent and independent variable and (for the independent variable) the different treatments. An example could be:

Table 5: Experiment layout – main experiment

	Task					
	Easy			Hard		
Language	1	2	3	1	2	3
Java	G1	G2	G3	G4	G5	G6
Groovy	G7	G8	G9	G10	G11	G12
Smalltalk	G13	G14	G15	G16	G17	G18

- **Dependent variables**

- Time until completion: It was measured how much time it took the participants to finish the tasks (see section 9.3)
- Number of errors: It was measured how many errors each participant did during each task (see section 9.3).

- **Independent variables**

- Programming Language: Java, Groovy, and Smalltalk
- Programming tasks
 - * Easy tasks: three easy tasks
 - * Hard tasks: three hard tasks

It might make sense say here some more word about the actual layout you have chosen. For example, if you decided to do a crossover trial, you can say something about it here (why you felt that this was necessary, etc.). In case, only some variables are measured within-subject (i.e. as a crossover trial) it makes sense to speak about it here as well.

Each dependent variable was measured using a between-subject design, i.e. the layout is a 3x2x3 between-subject experiments resulting into 18 different (independent) groups (see Table 5).

9.6 Experimental protocol

Describe here how the experiment should be executed. Say for each group what has to happen. This protocol is a step by step manual when you have to pass things to subjects, when you start and stop measuring and what kind of interaction is permitted to take place between the experimenter and the subject.

9.7 Experiment execution

In this subchapter you write down what was actually happened, i.e. how you collected the data. You should mention how many participants participated in the experiment and when they participated. You should give a description of

Table 6: Number of subjects per groups

Language	Task					
	Easy Task			Hard Task		
	1	2	3	1	2	3
Java	G1 (2)	G2 (2)	G3 (2)	G4 (2)	G5 (2)	G6 (2)
Groovy	G7 (2)	G8 (2)	G9 (2)	G10 (2)	G11 (2)	G12 (2)
Smalltalk	G13 (2)	G14 (2)	G15 (2)	G16 (2)	G17 (2)	G18 (2)

those participants and the process how you found them. In case you haven't said it before, you should mention how you assigned the participants to different groups.

You should say something about the location where the data collection took place. You should probably mention the machines it required to run the experiment (notebook?) or in case you do not know the machines (because it was a remote experiment) you should mention it as well.

Finally, give a table that mentions how the many subjects were assigned per group (see Table6) . In case, there are different groups of subject (such as novices/students/experts), say this in the table as well.

9.8 Results

The results are presented by first describing the data (raw measurements and descriptive statistics) followed by the statistical analysis.

9.8.1 Measurements and Data Description

First, you should give the raw measurements in case it is possible. If you measured 36 subjects, one measurements per subject, you can easily add a table here. Note: one of the main goals of writing down the reported data is to give readers the chance to recompute your results. This is really, really important. Just in case the measured data is too much, add a corresponding table to the appendix. And in the rare case that the numbers would fill countless pages, add the data to an external storage – and your thesis should always be delivered with this storage.

For each treatment you should write down the following measured data:

1. the minimum value
2. the maximum value
3. the arithmetic mean

Table 7: Descriptive statistics of measured data

	Java	Groovy	Smalltalk
Min	14514	14514	14514
Max	5432	5432	5432
M	5435	5435	5435
Median	5432	5432	5432
SD	123	123	123

4. the median

5. the standard deviation

There are situations where some of these numbers do not make much sense – make yourself clear what is needed by the reader in order to get an impression of the data. As a consequence, you might have other numbers such as average rank, etc.

Furthermore, give people a graphical impression of your data. A typical representation for it is a boxplot (see Figure 8). Try to describe the data as short as possible. Mention those things that are probably worth mentioning (such as huge differences in mean, or tendencies in the boxplot). For example, in the given example, it is valid to say “the boxplot gives the impression that the smalltalk with respect to time to completion takes more time”. But make sure that you just describe impressions – whether or not impressions are real is finally up to the statistical analysis.

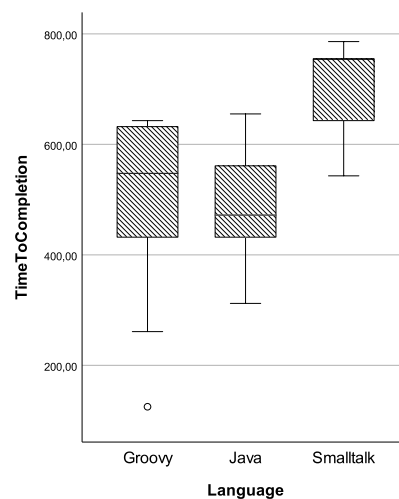


Figure 8: Boxplot for measured data

Make sure that you present boxplots for each hypothesis. If for example your hypothesis is just on time to completion and the treatments Java, Smalltalk, and Groovy, it is enough to have just one boxplot.

This section has probably just 1-2 pages (without the raw measurements).

Table 8: Results – ANOVA

	df	p	η_p^2	Treatment	N	CI _{95%}	M
Language	2	<.001	.654	Overall	40	[123;532]	432
				Java	14	[123;532]	123
				Groovy	18	[123;532]	543
				Smalltalk	8	[123;895]	643
Task	1	.094	.021	Overall	80	[123;532]	432
				HelloWorld	40	[123;532]	432
				Filter	40	[123;532]	432
Language*Task	2	0.87	.005	Overall	40	[123;532]	432

9.8.2 Statistical Analysis

Here comes the most important part of your work. The statistical analysis provides evidence for your hypotheses. Start by mentioning, what statistics software you applied in order to analyze your data. Yes, this is important, too, because you probably will trust in the results of the program – and in case someone else recomputes the data with a different program and receives different results, there is the need to check whether the differences are just the result of different programs or whether someone did something wrong.

What you need to report is first the way how you analyzed the data, i.e. mention the statistical method that you apply. A short explanation of this method should be found in the background chapter 2. Note that the goal of this chapter is just to report statistical results – do not make any interpretation. When you write “huge differences”, etc. this is an indicator that you are close to an interpretation.

You will describe the results of the analysis using a number of tables. Table 8 looks like a typical table that illustrates the results of an ANOVA. It contains the degrees of freedom, the p-value, the effect size (in that case the η_p^2 and for each treatment (in addition to overall) the number of data sets (N), the confidence intervals and the mean.

There are situations where the presentation of the means and confidence intervals are not so nice – especially, when you have some variables. In such a situation, a presentation such as in Table 9 is more appropriate.

Note that the number of statistical values you get can be very high. Concentrate in this chapter on the main factors and outcomes. The rest of the numbers can be presented in the appendix – and you just mention specific numbers here (but do not forget to add the reference to the appendix).

Very often, interactions between variables play an essential role. Add interaction diagrams here and describe the kind of interaction (and do not forget to mention the corresponding numbers!). On case, you have a high number of

Table 9: Main Experiment – means and confidence intervals

		Task – $p=.55$ / $\eta_p^2=.654$								
		Easy Task				Hard Task				Overall
Language ($p=.04$ / $\eta_p^2=.654$)		1	2	3	Overall	1	2	3	Overall	
Java	N	32	32	32	32	32	32	32	32	32
	M	432	432	432	432	432	432	432	432	432
	CI	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]
Groovy	N	32	32	32	32	32	32	32	32	32
	M	432	432	432	432	432	432	432	432	432
	CI	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]
Smalltalk	N	32	32	32	32	32	32	32	32	32
	M	432	432	432	432	432	432	432	432	432
	CI	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]
Overall	N	32	32	32	32	32	32	32	32	32
	M	432	432	432	432	432	432	432	432	432
	CI	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]	[312,421]

factors, there are probably countless interactions that need to be taken into account. Again, concentrate here on the most important ones and put the rest into the appendix.

9.9 Threats to Validity

In the threats to validity chapter, you put everything that could have potentially influenced the answer of your original research question. You can use the classification of threats (see the book by Wohlin et al. [14]), but it is not necessary the case that you do the classification of internal, external, conclusion and construct threats. Keep in mind: in the end, the text needs to be read by someone.

Very often, the threats to validity are quite speculative and some are quite obvious. For example, it is clear that crossover trials potentially suffer from carry over effects. Make clear when you have seen indicators that some of the threats are real (and more important than others).

9.10 Summary and discussion

Each experiment you did gets its' own summary and discussion chapter. You start with a summary where the main points of the experimental design and execution are mentioned. Then you described briefly the main results of your analysis. I.e. you mention again the main results of the statistical analysis. Here, in your discussion you can say whether some effect is from your perspective high or very high – and you should also take into comparison to studies from the related work (see 3) section into account.

Now, start with the discussion. Emphasize the threats that are most important from your perspective. And say, what the actual interpretation of your experiment is (from your perspective). This section easily fills 1 page.

10 Summary and Discussion

Again, you summarize and discuss your work. But this time, it is the complete work, i.e. you need to summarize the whole approach you have taken into account (from motivation over preliminary experiments to main experiment). It is also ok, if you say 1-2 sentences again about the motivation.

Then describe the main results of your experiments. There are probably some repetitions with the discussion section of each experiment, but that is ok – here, the summary is probably not as detailed as in previous discussions (such as in section 9.10).

Now, you can discuss your work as a whole. The difference to the discussion section is, that you not only refer to the results of an experiment, but you also take the transition from one experiment to the next one into account.

The discussion part is permitted to be slightly more subjective than the other sections. But keep in mind that text is still not prose.

This chapter is probably 1.5 pages or more.

11 Conclusion

Now, the main thing. What exactly should be concluded from your work? Keep in mind that the discussion has been already written, so it is typically the case that the conclusion is not at most 1 page. This conclusion can give an outlook of this work: what could be the influence of this work?

Well - if you have written this section, you are very close to finish. You just have to put some things into the appendix.

12 How References Work

Do not make your own bibtex entries, just copy them from the web, make sure all relevant information are shown (e.g. [2])

References

- [1] Stefan Hanenberg. Faith, hope, and love: An essay on software science's neglect of human factors. *SIGPLAN Not.*, 45(10):933–946, October 2010.
- [2] Andreas Jedlitschka, Marcus Ciolkowski, and Dietmar Pfahl. Reporting experiments in software engineering. In Forrest Shull, Janice Singer, and Dag I. K. Sjøberg, editors, *Guide to Advanced Empirical Software Engineering*, pages 201–228. Springer London, London, 01 2008.
- [3] Natalia Juristo and Ana M. Moreno. *Basics of Software Engineering Experimentation*. Springer Publishing Company, Incorporated, 1st edition, 2010.
- [4] Kaijanaho Kaijanaho. *Evidence-based programming language design : a philosophical and methodological exploration*. University of Jyväskylä, Finland, 2015.
- [5] Barbara Kitchenham, Hiyam Al-Khilidar, Muhammed Ali Babar, Mike Berry, Karl Cox, Jacky Keung, Felicia Kurniawati, Mark Staples, He Zhang, and Liming Zhu. Evaluating guidelines for reporting empirical software engineering studies. *Empirical Software Engineering*, 13(1):97–121, 2008.
- [6] Andrew J. Ko, Thomas D. Latoza, and Margaret M. Burnett. A practical guide to controlled experiments of software engineering tools with human participants. *Empirical Softw. Engg.*, 20(1):110–141, February 2015.
- [7] Dawn J. Lawrie, Christopher Morrell, Henry Feild, and David W. Binkley. What's in a name? A study of identifiers. In *14th International Conference on Program Comprehension (ICPC 2006)*, 14-16 June 2006, Athens, Greece, pages 3–12. IEEE Computer Society, 2006.
- [8] Karl Raimund Popper. *The Logic of Scientific Discovery*. Routledge, 2002. 1st English Edition:1959, Original First Edition (German): *Logik der Forschung*, published 1935 by Julius Springer, Vienna, Austria.
- [9] Baishakhi Ray, Daryl Posnett, Premkumar Devanbu, and Vladimir Filkov. A large-scale study of programming languages and code quality in github. *Commun. ACM*, 60(10):91–100, September 2017.
- [10] Andreas Stefik and Stefan Hanenberg. The programming language wars: Questions and responsibilities for the programming language community. In *Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software, Onward! 2014*, pages 283–299, New York, NY, USA, 2014. Association for Computing Machinery.
- [11] Andreas Stefik and Stefan Hanenberg. Methodological irregularities in programming-language research. *IEEE Computer*, 50(8):60–63, 2017.

- [12] Andreas Stefik, Stefan Hanenberg, Mark McKenney, Anneliese Andrews, Srinivas Kalyan Yellanki, and Susanna Siebert. What is the foundation of evidence of human factors decisions in language design? an empirical study on programming language workshops. In *Proceedings of the 22nd International Conference on Program Comprehension, ICPC 2014*, pages 223–231, New York, NY, USA, 2014. Association for Computing Machinery.
- [13] S. Vegas, C. Apa, and N. Juristo. Crossover designs in software engineering experiments: Benefits and perils. *IEEE Transactions on Software Engineering*, 42(2):120–135, 2016.
- [14] Claes Wohlin, Per Runeson, Martin Hst, Magnus C. Ohlsson, Björn Regnell, and Anders Wessln. *Experimentation in Software Engineering*. Springer Publishing Company, Incorporated, 2012.

A Appendix

A.1 Experiment Summary

Add here for each experiment a table that briefly describes the experiment. Don't forget it. This experiment summary gives people a very, very quick overview – it really helps a lot.

Table 10: Experiment design summary - main experiment

A meaningful name for the experiment																																																																				
1. Short summary																																																																				
Hypotheses	H0 _{devTime} : No difference in development time between Java, Groovy, and Smalltalk					REJECTED																																																														
	H0 _{somethingElse} : No difference in something additionally between Java, Groovy, and Smalltalk					REJECTED																																																														
2. Layout & Design																																																																				
Dependent variables	1) Development Time 2) Something additionally																																																																			
Independent variables and treatments	1) programming language: Java, Groovy, Smalltalk 2) kind of task: easy, hard 3) task: 3 tasks																																																																			
Additional variables	—																																																																			
Hypotheses	H0 _{devTime} : No difference in development time between Java, Groovy, and Smalltalk					REJECTED																																																														
	H0 _{somethingElse} : No difference in something additionally between Java, Groovy, and Smalltalk					REJECTED																																																														
Experiment Layout	Between-subject on all independent variables																																																																			
Protocol	<table><tr><td rowspan="2">Language</td><td colspan="3">Task</td><td colspan="3"></td></tr><tr><td colspan="2">Easy Task</td><td></td><td colspan="2">Hard Task</td><td></td></tr><tr><td>Java</td><td>1</td><td>2</td><td>3</td><td>1</td><td>2</td><td>3</td></tr><tr><td></td><td>G1</td><td>G2</td><td>G3</td><td>G4</td><td>G5</td><td>G6</td></tr><tr><td></td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td></tr><tr><td>Groovy</td><td>G7</td><td>G8</td><td>G9</td><td>G10</td><td>G11</td><td>G12</td></tr><tr><td></td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td></tr><tr><td>Smalltalk</td><td>G13</td><td>G14</td><td>G15</td><td>G16</td><td>G17</td><td>G18</td></tr><tr><td></td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td><td>(2)</td></tr></table>						Language	Task						Easy Task			Hard Task			Java	1	2	3	1	2	3		G1	G2	G3	G4	G5	G6		(2)	(2)	(2)	(2)	(2)	(2)	Groovy	G7	G8	G9	G10	G11	G12		(2)	(2)	(2)	(2)	(2)	(2)	Smalltalk	G13	G14	G15	G16	G17	G18		(2)	(2)	(2)	(2)	(2)	(2)
	Language	Task																																																																		
		Easy Task			Hard Task																																																															
	Java	1	2	3	1	2	3																																																													
		G1	G2	G3	G4	G5	G6																																																													
		(2)	(2)	(2)	(2)	(2)	(2)																																																													
	Groovy	G7	G8	G9	G10	G11	G12																																																													
		(2)	(2)	(2)	(2)	(2)	(2)																																																													
	Smalltalk	G13	G14	G15	G16	G17	G18																																																													
		(2)	(2)	(2)	(2)	(2)	(2)																																																													
- Answer questions that subject has concerning experiment execution																																																																				
- Open IDE (Eclipse) with programming task																																																																				
- start time measurement																																																																				
- stop time measurement once all test cases are fulfilled by subject																																																																				

Table 11: Experiment execution summary - main experiment

3. Execution							
Subjects assignments / Subjects	Random assignment 36 Subjects (2 subjects for each of the 18 groups) 30 subjects students at University of Duisburg-Essen 6 subjects professional developers						
Location	Individual meetings at University of Duisburg–Essen						
Time	November and Dezember 2020						
4. Results							
	df	p	η_p^2	Treatment	N	CI _{95%}	M
Language	2	<.001	.654	Overall	40	[123;532]	432
				Java	14	[123;532]	123
				Groovy	18	[123;532]	543
				Smalltalk	8	[123;895]	643
Task	1	.094	.021	Overall	80	[123;532]	432
				HelloWorld	40	[123;532]	432
				Filter	40	[123;532]	432
Language*Task	2	0.87	.005	Overall	40	[123;532]	432
Main interpretation	Some brief sentences what the experiment tell us.						

A.2 Handling Tables and Table Formats in Lyx

In case data collection is part of the thesis, there is a high chance that you are using tables in your thesis. Depending on what kind of data is collected (and how much data is required), there are different kinds of tables and table format. Although Lyx is much more comfortable than for example Latex, adding or formatting tables is still not that comfortable than for example in Word. However, typically you just need some standard ways for tables that you can just copy and paste from here.

```
1 public class dummy {
2 }
```

A.2.1 Standard Table

A standard table can be found in Table 12. There are

A.2.2 Long Table or Figure Descriptions

It is a good idea to have table or figure descriptions that are self-contained, i.e. that all text that is necessary to understand what is in the ta-

Table 12: Centralized table with dynamic column size

FirstColumn	SecondColumn
123	123
123	123123

Or maybe you prefer a slightly different format where the first row is not emphasized?

FirstColumn	SecondColumn
123	123
123	123123

Table 13: Centralized table with row breaks (requires fixed sizes column size)

This column has a fixed size. This permits you also to have you own line break (just by entering <return>)	This one has not such a fixed size => everything in one row

Table 14: Sometimes it is nicer to resize the table so that is matches on a page

a	b	c	d	e	d	f	f	f	f	d	s		a	s	d	f	ds	a	sd	f	d	s	a	s	d	f	ds	a	s
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds
as	d	sdf	d	fs	df	df	d	f	df	df	df	s	d	fd	s	fd	s	a	sdf	d	sa	sdf	df	ds	df	d	sfd	s	ds

ble or figure. However, a long descriptions does not look nice in the list of figures or list of table. Hence, you should chose a short description. See <https://tex.stackexchange.com/questions/263256/caption-of-figure-in-list-of-figures-in-lyx>

A.2.3 Highlighting Stuff

For highlighting text, use the `\hl` command from the soul package, for example:

Alle sagen: „**Stefan is doof**“.

In order to highlight cells or rows in a table, use the command `\rowcolor` or `\cellcolor`. See for example:

ASD	sad	ASD	a	sdA
SDa	sdA	DS	asd	A
DSadsADS	adsADSadsADSads	ASD	ads	ASDasdADSadsASDads
adsASDasdASDa	sd	ADS	ads	ADSasdADSads
adsADSasdASDads	A	DSa	ds	ADS

Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe alle Stellen, die ich aus den Quellen wörtlich oder inhaltlich entnommen habe, als solche kenntlich gemacht. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Essen, 10.10.2020